School of Informatics & Cybersecurity - Assessment Submission Coversheet

Student Information

Student Number:	B00163194
Student Name:	Matas Zostautas

Module Information

Module Title:	Web Framework Development
Module Code:	COMP H3037

Assessment Information

Assessment Title:	Project

Declaration

☒ I confirm that I have read and understood the University's Academic Integrity Policy.

https://www.tudublin.ie/explore/about-the-university/academic-affairs/academic-quality-assurance-and-enhancement/academic-integrity/

☒ I have completed the TU Dublin AI Driving License theory test.

https://www.tudublin.ie/explore/about-the-university/academic-affairs/lta/lta-resources/artificial-intelligence-driving-license/

☒ I declare that this assessment submission is my own work.

☒ I confirm that I have read and understood the AI traffic light system used as part of this assessment brief and I have adhered to the level of AI usage allowed for this assessment.

Student Signature: Matas Zostautas

Date: 27/04/2026

Web Framework Development Project:

Theme: Education

Use Case: Inventory (Item – Categories)

Submitted by: Matas Zostautas, B00163194

Table of Contents

School of Informatics & Cybersecurity - Assessment Submission Coversheet 1
Student Information 1
Module Information 1
Assessment Information 1
Declaration 1
1. Business Understanding 3
Business Requirements 4
2. Use Cases 4
3. Database Design 8
4. Django Framework Implementation 10
5. Project Testing 19
6. Media Sources 22

1. Business Understanding

Assetra is a web-based school inventory management system designed to show how an educational institution tracks, lends, and manages assets such as laptops, projectors, laboratory equipment, textbooks and more. Schools often rely on spreadsheets or paper logs to keep track of which items have been loaned to which student and when those items are due to be returned. This approach could have errors, difficult to keep track, and difficult to always see stock levels.

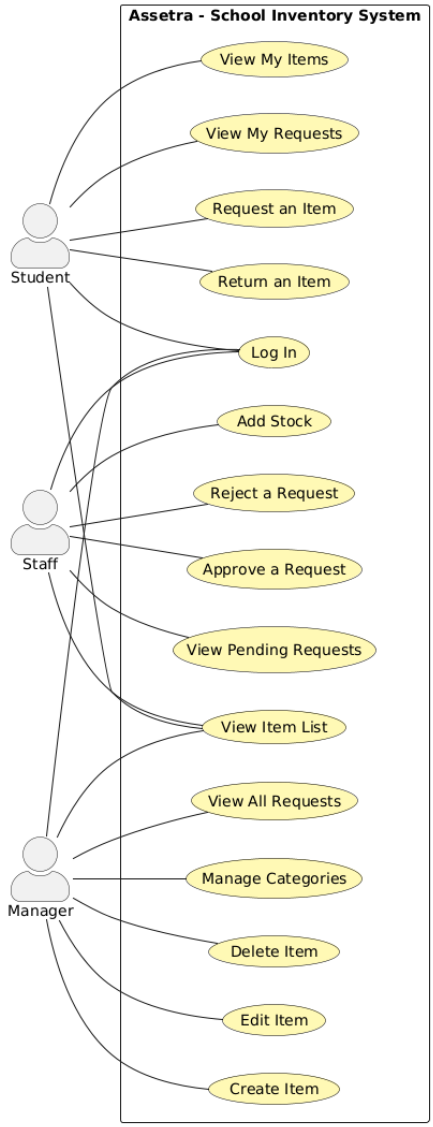
The Assetra platform will address all these problems by providing a centralised role-based web application where students or users can browse available items and request for these items, followed by a staff member reviewing and perform an action on those requests. Managers can maintain the catalogue of items and categories. All transactions are logged to the database, giving the school full visibility of inventory changes at any point.

Business Requirements

- The system must provide role-based authentication and permission which covers Students, Staff and Managers as non-admin roles.
- Students must be able to browse the inventory, request items that are currently in stock, view the status of their requests, and return any items that have been approved.
- Staff must be able to view pending requests and either approve or reject them, with approvals immediately and automatically decrementing stock levels.
- Managers must be able to perform full CRUD operations on items and categories and view a complete history of requests made across the systems life.
- Out-of-stock items must not be able to be requested, and approvals must be blocked if a particular stock has run out before someone could be approved for it.
- Every request must have a status attached. Such status includes Pending, Approved, Rejected, or Returned, allowing the staff to be able to trace each item loan lifecycle.

2. Use Cases

The system supports 3 key roles: Staff, Student, and Manager. There are 2 other roles that do not play a significant role in the system. One role is the user role (basic) and is very restricted (can only view items) whereas the admin role may do and overlook everything.



General Use Cases

Use Case1: Log In
Any authenticated user can log in (Student, Staff, Manager, Admin)
The user enters their username and password. On success, Django will authenticate them and they will be redirected to the role appropriate landing page (Staff, Dashboard, Manager Dashboard, Student Home page)

Use Case 2: View List Item
All users (even non authenticated users/visitors) can view the items.
Displays every item in the inventory, with its name, description, quantity, stock, status (Good, Low, Out), and category. Action buttons are dependent on the role.

Student Use Cases

Use Case 3: Request an Item
The student that is logged in and items quantity is not 0. The student clicks the 'Request' button on the item list, and a new request record is created with status 'PENDING' and is sent for staff review.

Use Case 4: View My Requests
Displays all requests submitted by the logged in student with their status (Pending, Approved, Rejected, Returned) and the date the request was made.

Use Case 5: View My Items
Filters the student's requests to show only those currently approved, such as items in the student's possession.

Use Case 6: Return Item
The request status is 'APPROVED' and the user is the requester. The items stock quantity is incremented by one and the request status is 'RETURNED'.

Staff Use Cases

Use Case 7: View Pending Requests
Displays every request currently in 'PENDING' status with the requesting user and the item requested.

Use Case 8: Approve a Request
Request must be pending and item stock must be greater than 0. Decrements the item quantity by one and sets the request status to 'APPROVED'. If the item is out of stock at the time of approval, the operation is rejected with an error message.

Use Case 9: Reject a Request
Marks a pending request as 'REJECTED' without affecting stock levels.

Use Case 10: Add Stock
Increments the quantity of a selected item by one, used when physical stock is replenished.

Manager Use Cases

Use Case 11: Create Item
Opens a form where the manager can add a new item (name, description, quantity, category) to the inventory.

Use Case 12: Edit Item
Edit an item through a form. When edits are made and saved, the items information in the database is updated.

Use Case 13: Delete Item
After confirming to delete an item, the item is permanently removed and any associated request made for the item are also deleted.

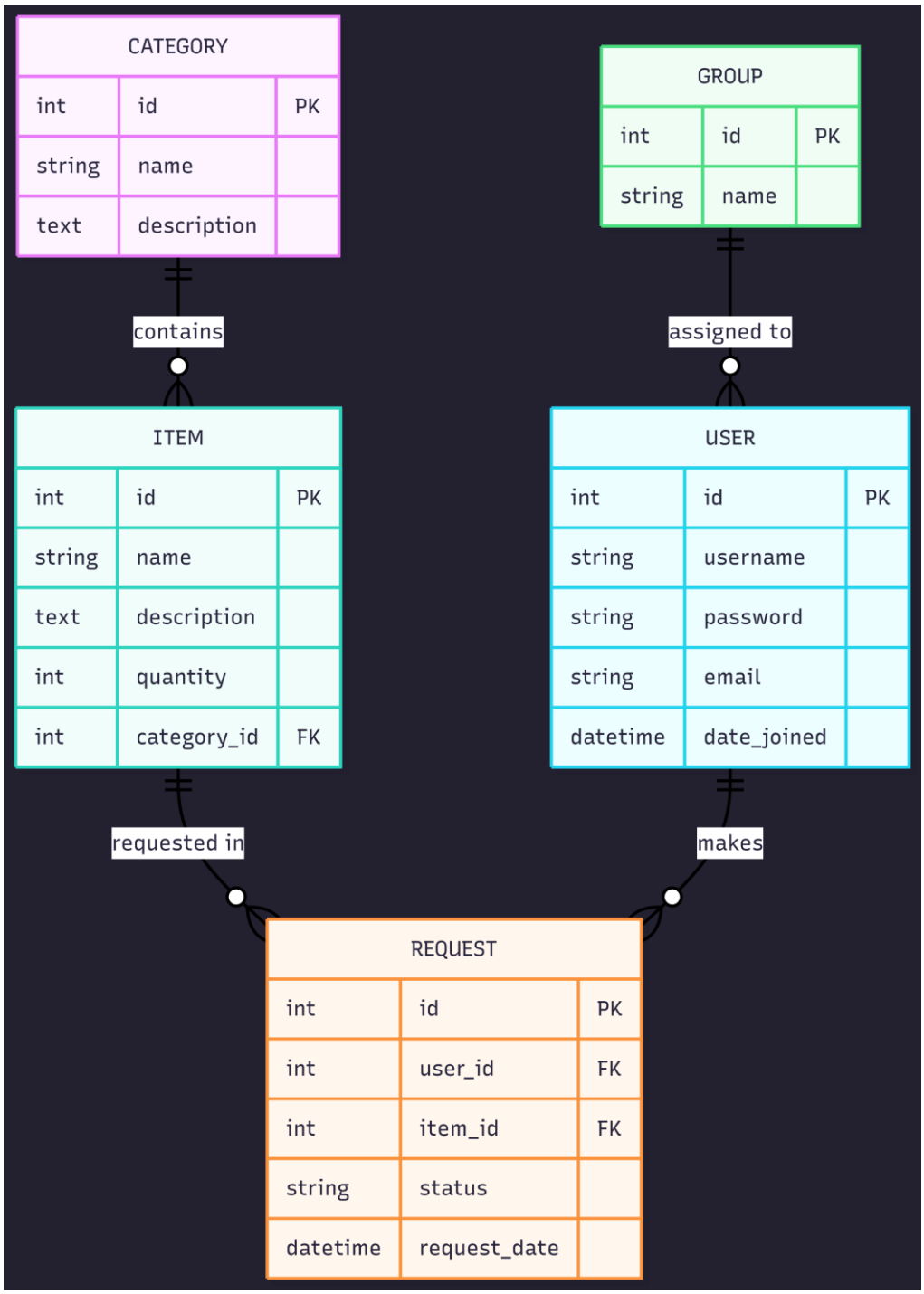
Use Case 14: Manage Categories
Manage the listing, creation and editing of Category records which are used to categorise items (Electronics, Laboratory Equipment, Books, Sports Equipment and more if needed)

Use Case 15: View All Requests Made
Provides a view of every request in the system with user, item, status, and date.

Use Case 16: Requests as CSV
The manager can export the full request history as a CSV file. This is done with a click of a button, and the manager can see every request with the user, item, category, status and date formatted as a spreadsheet.

3. Database Design

The Assetra database is built around **five entities**: Category, Item, Request, User, and Group.



Entities and Attributes

Category
id(PK) - Primary key for each individual category
name(string) – The category name (Electronics, Books)
description(string) – A description of what the category contains

Item
id(PK) – Unique primary key for each individual category
name(string) – The name of the item
description(string) - A description of the item
quantity(int)- The current stock level available
category_id(FK – Category) – The category the item belongs to

Request
id(PK) – Primary key for each request
user_id(FK – User) – The student who made the request
item_id(FK – Item) – The item being requested
status(string) – Pending, Approved, Rejected, or Returned
request_date(datetime) – Timestamp of when the request was made

User and Group (Django Admin)
User holds credentials and profile info. Each user belongs to a group in the system The group defines the role. This can be Admin, Staff, Student, Manager or a basic User.

Relationships
Category (1 to Many) Item: a category can contain many items and each items belong exactly to one category.
User (1 to Many) Request: a user can make many requests.
Item (1 to Many) Request: an item can be requested many times.
User (Many to 1) Group: many users can be in one group.

Database Fixtures

To ensure the database is always populated with consistent data on each fresh deployment, a fixture file (data.json) is made in the project root. The fixture contains data for every entity in the system, which includes 5 Categories, 12 Items spread through these categories, 7 Requests in different lifecycles (Pending, Approved, Rejected, Returned), and 6 Users assigned to 4 different Groups (Admin, Staff, Manager, Student, with an additional basic User). The fixture is loaded on every

Railway deployment via a Procfile, which runs python manage.py loaddata data.json immediately after migrations.

4. Django Framework Implementation

Assetra is built using Django's Model-View-Template (MVT) architecture.

Model (M)

The three models are defined in inventory/models.py

```
class Category(models.Model):
    name = models.CharField(max_length=100)
    description = models.TextField()

    def __str__(self):
        return self.name
```

```
class Item(models.Model):
    name = models.CharField(max_length=100)
    description = models.TextField()
    quantity = models.IntegerField()
    category = models.ForeignKey(Category, on_delete=models.CASCADE)

    def __str__(self):
        return self.name
```

```
class Request(models.Model):
    STATUS_CHOICES = [
    ('PENDING', 'Pending'),
    ('APPROVED', 'Approved'),
    ('REJECTED', 'Rejected'),
    ('RETURNED', 'Returned'),
]
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    item = models.ForeignKey(Item, on_delete=models.CASCADE)
    status = models.CharField(max_length=10, choices=STATUS_CHOICES, default='PENDING')
    request_date = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.user} requested {self.item}"
```

There would have been a User and Group model here also however Django already has that built in the auth system, so I did not need to make this myself. Each new request has a date stamp with auto_now_add=True and automatically defaults to a 'PENDING' status.

View (V)

Views are implemented as function-based views in inventory/views.py. Each view has its own unique function and is protected with the @login_required decorators and for role-specific views, simple helper functions check to see which role the logged in user is.

```
def is_staff(user):
    return user.groups.filter(name='Staff').exists()

def is_student(user):
    return user.groups.filter(name='Student').exists()

def is_manager(user):
    return user.groups.filter(name='Manager').exists()

def is_admin(user):
    return user.groups.filter(name='Admin').exists() or user.is_superuser
```

To make the database a bit safer, invalid or unsafe database lookups were handled by get_object_or_404() which means that invalid ID's return with an error response instead of crashing the entire server. Django's messages framework is also implemented and is used to give the user feedback after every action, such as approving a request or successfully returning an item.

The home view is the central view. It essentially checks the logged in user's role and redirects them to the appropriate dashboard.

```python
@login_required
def home(request):
    if is_admin(request.user):
        return redirect('admin_dashboard')
    if is_staff(request.user):
        return redirect('staff_dashboard')
    if is_manager(request.user):
        return redirect('manager_dashboard')
    return render(request, 'inventory/home.html')
```

For the managers create/edit views, I used Django's ModelForm class which is defined in inventory/forms.py. A ModelForm allowed me to easily generate form fields meaning I did not have to write raw HTML <input> tags.

Template (T)

All templates are in inventory/templates/inventory/ and extend off a single shared base template (base.html).

- The base template contains:
- The site navigation bar with a role-coloured badge showing the logged in user and their assigned role.
- The message area (Django's messages Framework)
- Bootstrap CSS + JS and Bootstrap Icons links

Each individual page templates (items.html, manage_requests.html, manager_dashboard.html and more) extends base.html and simply fills in the {% block content %} block. This means that all these pages are inheriting the code from the base template and that means I only had to write the navbar and footer once.

Role-based page rendering is handled inside the templates using Django's template tags. For example, the "Actions" columns on the items page shows different buttons depending on who is currently logged in.

```
{% if role == "Student" %}
    {% if item.quantity > 0 %}
        <a href="/request/{{ item.id }}/" class="btn btn-primary btn-sm">
            <i class="bi bi-send me-1"></i>Request
        </a>
```

Assetra

student1 · Student | Logout

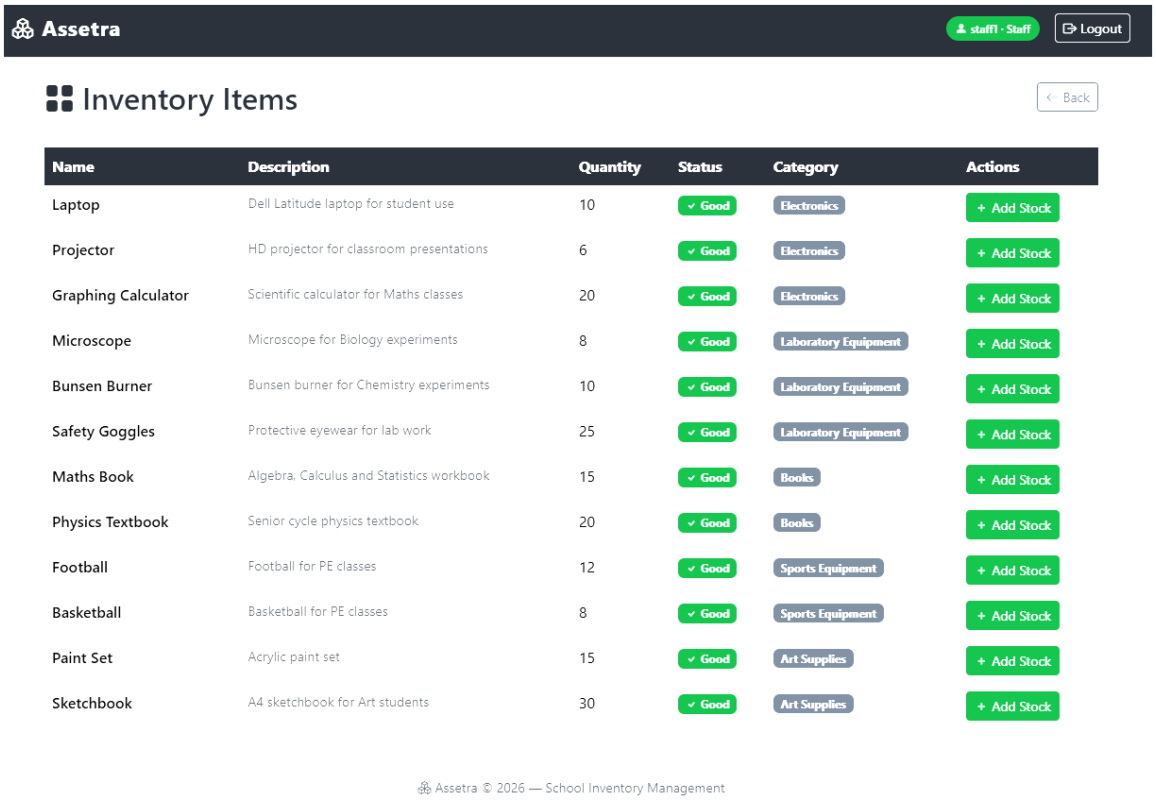
Inventory Items

Back

Name	Description	Quantity	Status	Category	Actions
Laptop	Dell Latitude laptop for student use	10	Good	Electronics	Request
Projector	HD projector for classroom presentations	6	Good	Electronics	Request
Graphing Calculator	Scientific calculator for Maths classes	20	Good	Electronics	Request
Microscope	Microscope for Biology experiments	8	Good	Laboratory Equipment	Request
Bunsen Burner	Bunsen burner for Chemistry experiments	10	Good	Laboratory Equipment	Request
Safety Goggles	Protective eyewear for lab work	25	Good	Laboratory Equipment	Request
Maths Book	Algebra, Calculus and Statistics workbook	15	Good	Books	Request
Physics Textbook	Senior cycle physics textbook	20	Good	Books	Request
Football	Football for PE classes	12	Good	Sports Equipment	Request
Basketball	Basketball for PE classes	8	Good	Sports Equipment	Request
Paint Set	Acrylic paint set	15	Good	Art Supplies	Request
Sketchbook	A4 sketchbook for Art students	30	Good	Art Supplies	Request

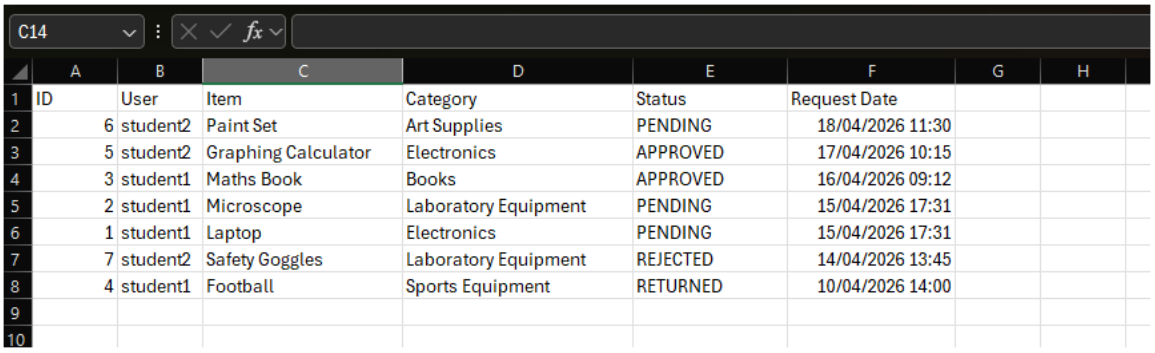
Assetra © 2026 — School Inventory Management

```
{% if role == "Staff" %}
    <a href="/add_stock/{{ item.id }}/" class="btn btn-success btn-sm">
        <i class="bi bi-plus me-1"></i>Add Stock
    </a>
```



Advanced Feature

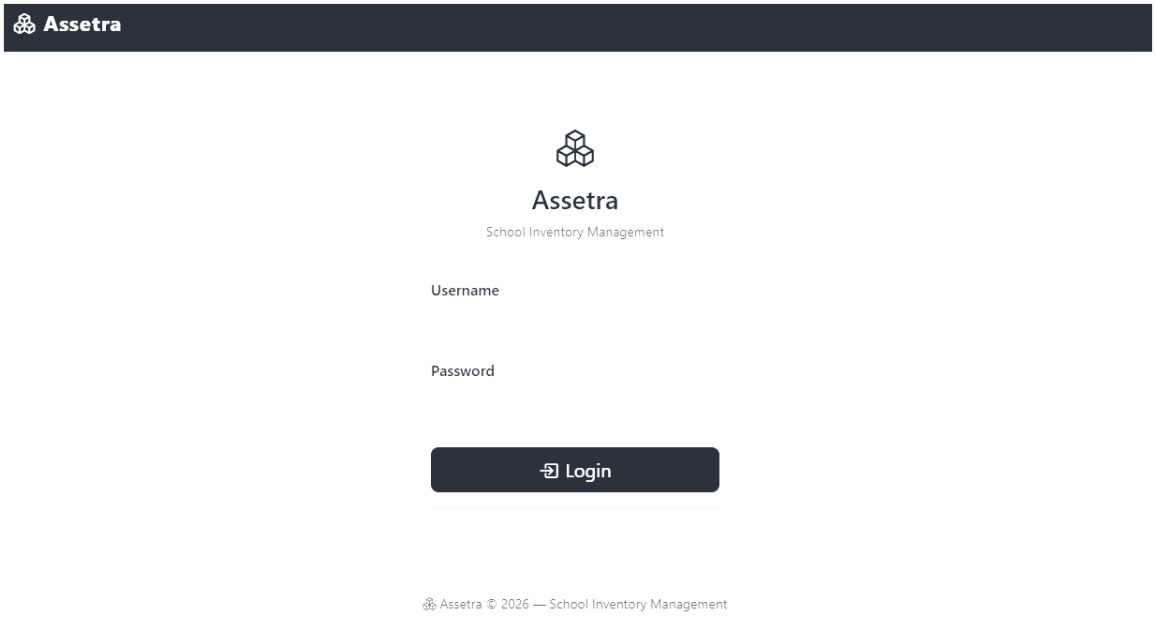
As an advanced feature beyond standard CRUD, Assetra implements automated CSV reporting for managers. This adds real-world value for school administrators who need request data for audits, end-of-term reports, and asset purchasing decisions.



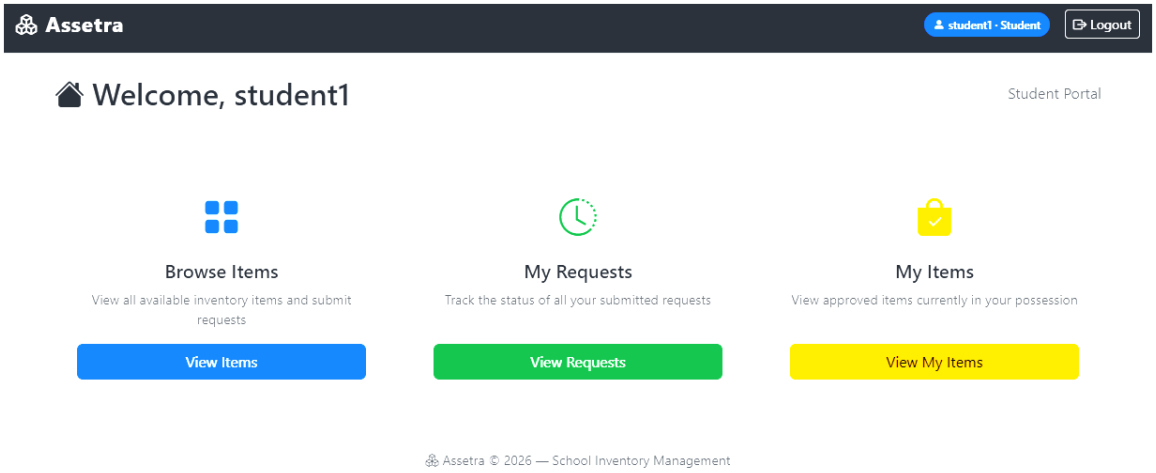
ID	User	Item	Category	Status	Request Date
6	student2	Paint Set	Art Supplies	PENDING	18/04/2026 11:30
5	student2	Graphing Calculator	Electronics	APPROVED	17/04/2026 10:15
3	student1	Maths Book	Books	APPROVED	16/04/2026 09:12
2	student1	Microscope	Laboratory Equipment	PENDING	15/04/2026 17:31
1	student1	Laptop	Electronics	PENDING	15/04/2026 17:31
7	student2	Safety Goggles	Laboratory Equipment	REJECTED	14/04/2026 13:45
4	student1	Football	Sports Equipment	RETURNED	10/04/2026 14:00

Extra Screenshots of Web Framework

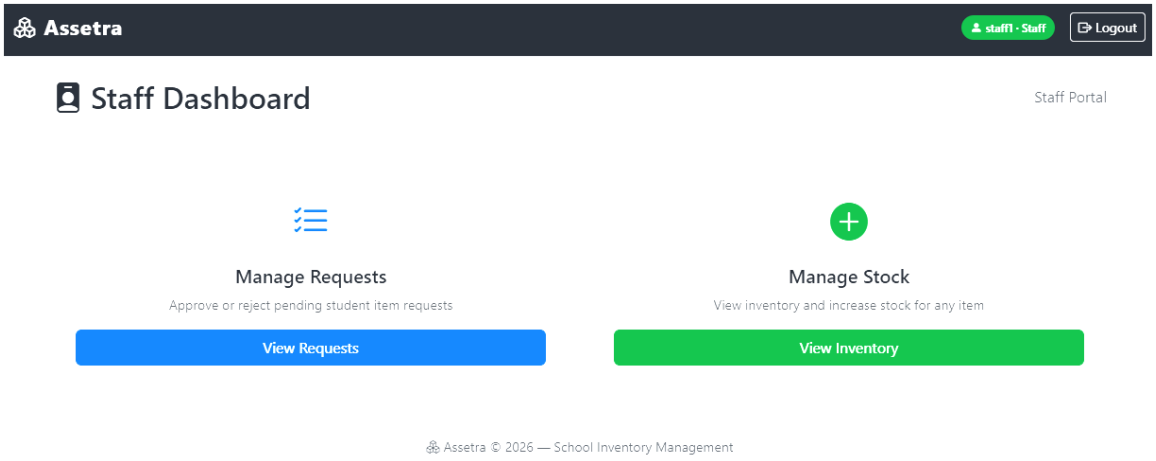
Login page



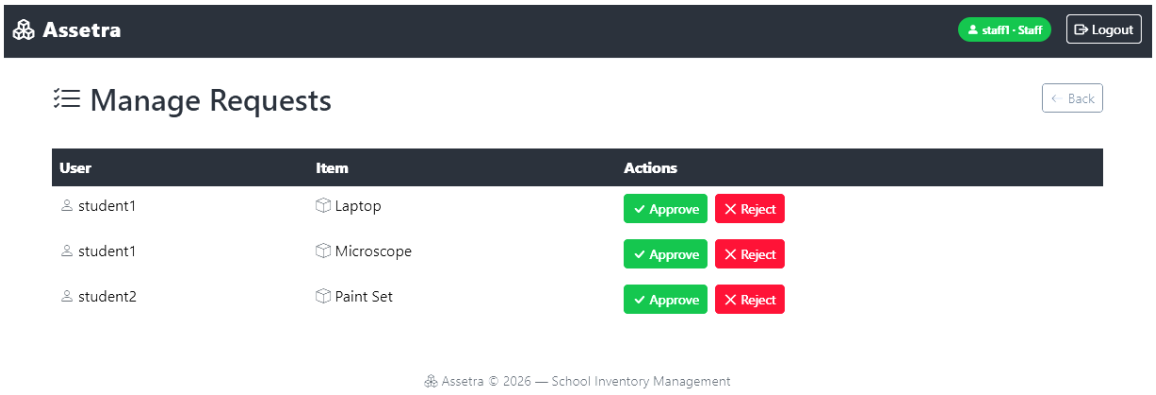
Student Dashboard



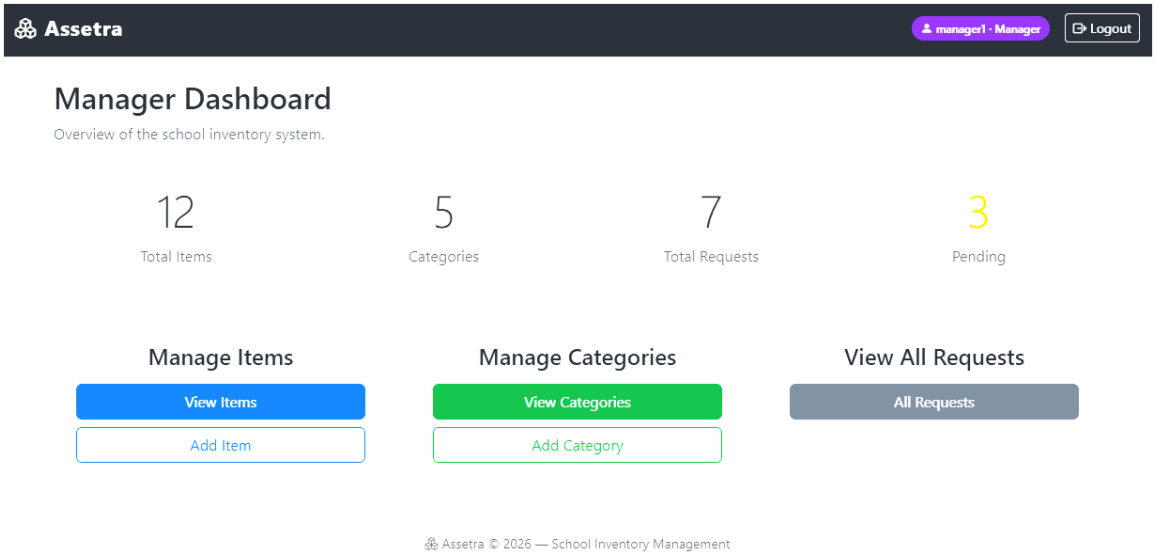
Staff Dashboard



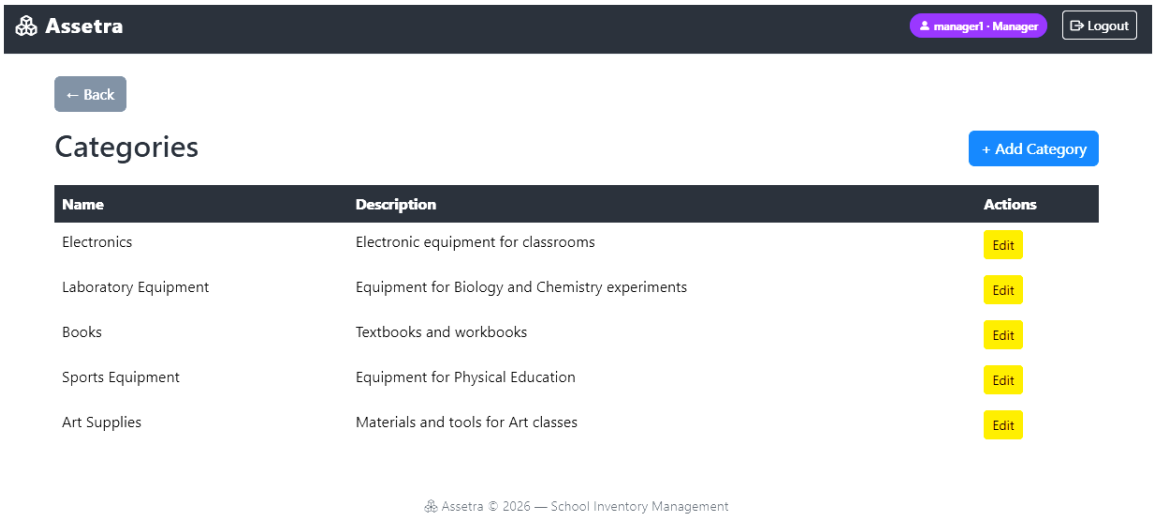
Staff Manage Requests



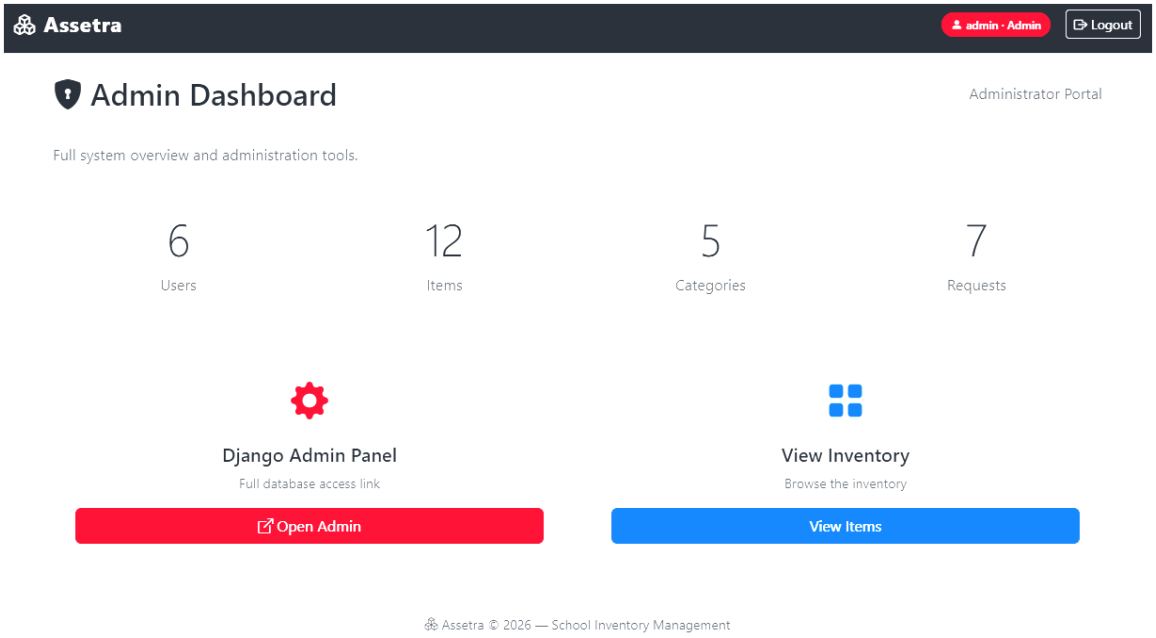
Manager Dashboard



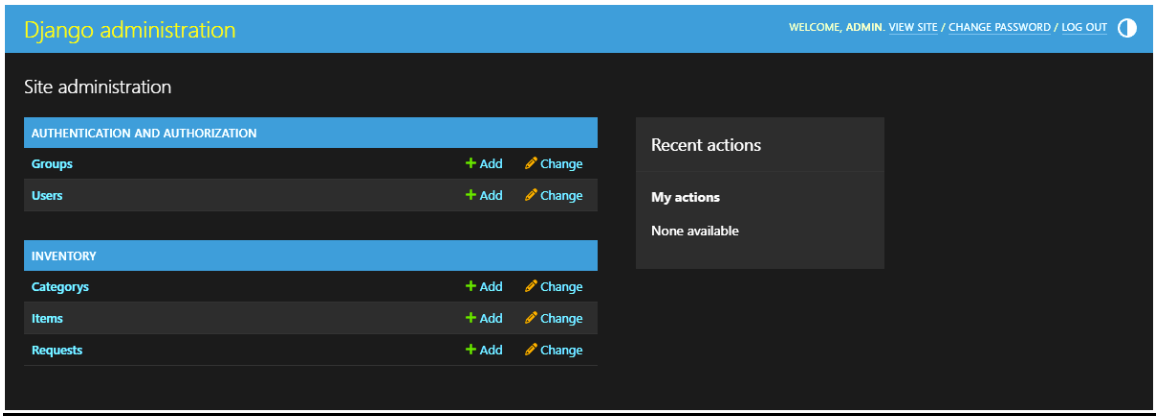
Manager Edit Categories



Admin Dashboard



Admin System Overview



5. Project Testing

Automated testing in Assetra is implemented in inventory/tests.py using Django's built in test framework. The testing strategy covers all three types of test types required for the project; unit tests, integration tests, and use case tests.

Test Cases Implemented

Unit Tests:

test_item_quantity_can_be_updated – creates an item, changes the quantity, saves it, and verifies if the changes stay in the database.

test_user_assigned_to_correct_group – creates a manager user and verifies if the user is correctly assigned to the Manager group.

Integration Tests:

test_item_list_is_visible – sends a GET request for a item to check if it is there and visible

test_student_cannot_access_manage_request – logs in as a student, attempts to access the staff only page, and checks if they are redirect back to home

test_login_required_to_make_request – test to see if the @login_required decorator is functioning correctly ensuring only authenticated requests can be made.

Use Case Tests:

test_student_can_request_item – logs in as student, requests an item, and verifies if the request was made in the database.

test_staff_can_approve_request- logs in as staff, approves a pending request, and verifies that the request status has changed from PENDING to APPROVED

test_manager_can_create_item_via_form – logs in as Manager, submits the Add Item form with valid data and verifies if the new item has been saved in the database. This test is for the form filling validation.

Tests Running

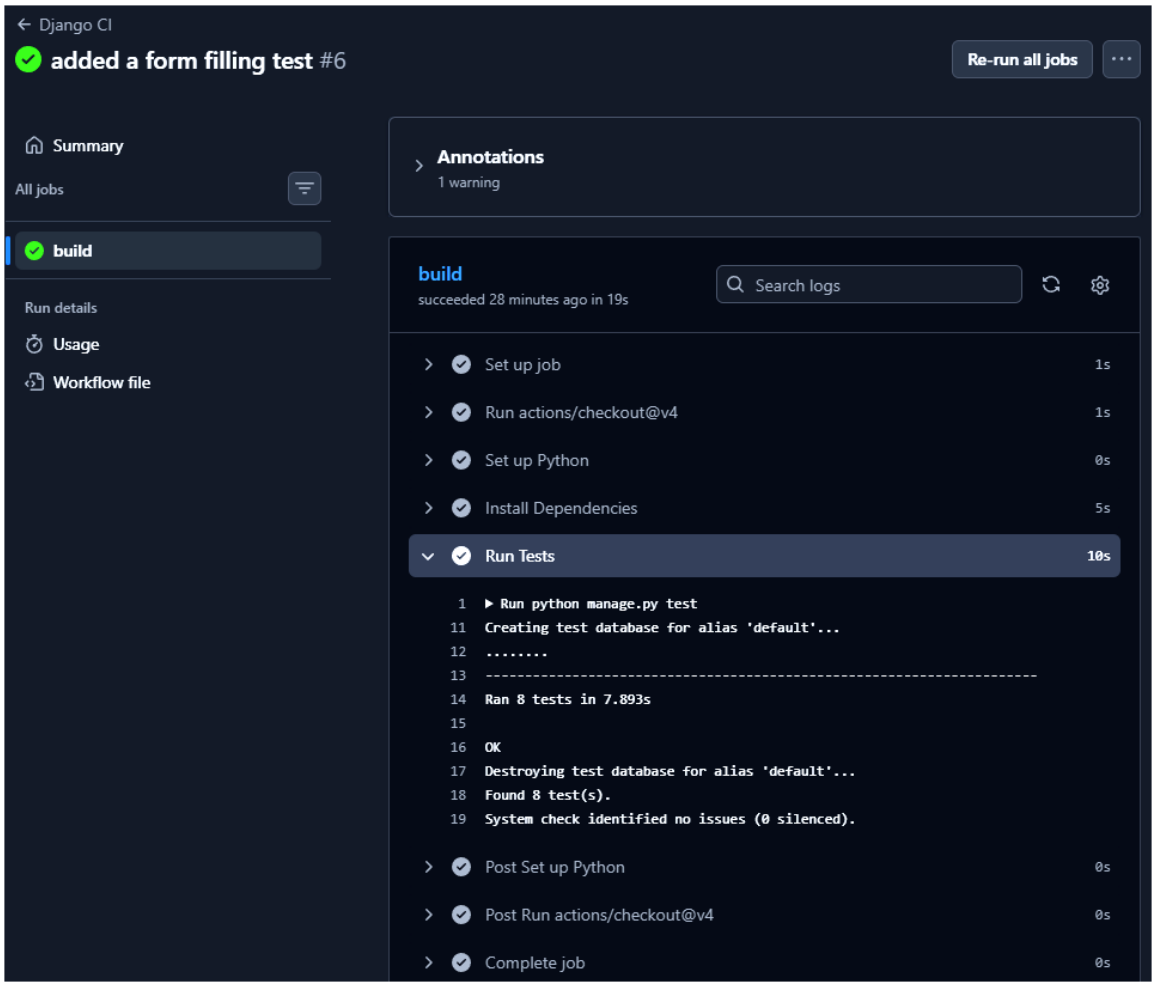
```
(venv) PS C:\Users\matas\Documents\GitHub\school_inventory> py manage.py test inventory
Found 8 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
........
----------------------------------------------------------------------
Ran 8 tests in 6.229s

OK
Destroying test database for alias 'default'...
```

All eight automated tests passed against the current codebase, showing and confirming that the model logic, role-based access controls, and core user flows are functioning correctly.

Continuous Integration & Continuous Deployment

The project also implements Continuous Integration (CI) using GitHub Actions. This workflow is done in .github/workflows/django.yml and is triggered automatically every time there is a push to the main branch. This ensures that all the tests are passing after any change, which can help catch issues like missing dependencies or any other minor issues before they reach production and deployment. This CI pipeline is also paired with Continuous Deployment(CD) provided by Railway, which automatically builds and redeploys the application on every push to main. With these two pipelines together, every commit is tested and if the tests pass, the commit can go on to being deployed to production.


Django CI
added a form filling test #6
Re-run all jobs
Summary
All jobs
build
Run details
Usage
Workflow file
Annotations
1 warning
build
succeeded 28 minutes ago in 19s
Search logs
Set up job 1s
Run actions/checkout@v4 1s
Set up Python 0s
Install Dependencies 5s
Run Tests 10s
1 ▶ Run python manage.py test
11 Creating test database for alias 'default'...
12 .....
13 ----------
14 Ran 8 tests in 7.893s
15
16 OK
17 Destroying test database for alias 'default'...
18 Found 8 test(s).
19 System check identified no issues (0 silenced).
Post Set up Python 0s
Post Run actions/checkout@v4 0s
Complete job 0s

6. Media Sources

Use Case Diagram Tool

https://editor.plantuml.com/uml/SoWkIImgAStDuIhEpimhI2nAp5L8J2x9BmekgSn9LKWiJotEpqtbiWejJYsoKj3LjLFG038rkPHy4pFp51nJSZFpb1GIYo4iBYMWd4991b1VGK5EPd9YIMO-H1gY2Qm8OWY4YQPKayiXDIy5v1W0

Bootstrap CSS for Design:

https://getbootstrap.com/docs/5.3/getting-started/introduction/

https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css

Bootstrap CSS for Icons:

https://icons.getbootstrap.com

https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.css

ERD Diagram Tool:

https://mermaid.live/edit#pako:eNqtU1Fv2jAQ_ivWPVMUisE0b1OXVlVVtaPwsAkJefGRuiV2ajtbGeW_zzYJMIamTZqf4u----7z3WUNuRYIKaD5KHlheDlTxJ_r8f30gby_n53pNZk-ZmOSkhlwa2WhUBCnZ7AlxljDG2efptnjJFJL_oK2JV1-mGTX9-PPLfFmkt1FVr4Mkgu5p8bQCT2DrzVa52tLFbiHNtfbSzhSOSIFebjdQ9YZqQqieIlbcNNmR-9_lVxbNHuBg0Dl_X_XRvwWwJLL5R79qvUSuSLSzm1doQmCp6OOLxYnIzx38tuBA8EdOlli_Jg_a-kHc_TAXd__sUPhOHxzRKDNjayc1OpIOs7pv8m2Cq81V0661a9o7h9YaLOa-wJXt0dG2iX5s5cAhZ4fSuyoDstjvPHup-Fqe6LnzTrOA9AYgg4URgpInamxAyUavwL-CtHZDNwT-kZAWGbBzUtY4pBTcfVF67JNM7ouniBd8KX1t7oKBZo_c4caVALNpa6Vg5T2ogaka3iDtNdjXcrYRcIGfTqkyagDq4Ced3tDNuyPLihLKE3YpgM_YtWkywYJHZ0PhrSf-Jw-2_wE0nA1CQ

Bootstrap JS Simple Interactivity

https://getbootstrap.com/docs/5.3/getting-started/introduction/

https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js

Django ModelForm

https://docs.djangoproject.com/en/6.0/topics/forms/modelforms/

Django auto_now_add

https://docs.djangoproject.com/en/6.0/ref/models/fields/

Django get_object_or_404()

https://docs.djangoproject.com/en/6.0/topics/http/shortcuts/

Django messages Framework

https://docs.djangoproject.com/en/6.0/ref/contrib/messages/

Static File Fix (Django Admin CSS file)

https://stackoverflow.com/questions/54685019/when-exactly-do-i-need-staticfiles-urlpatterns

https://docs.djangoproject.com/en/5.0/howto/static-files/

Gunicorn Installation

https://gunicorn.org

Continuous Integration (tests running automatically after every git commit)

https://docs.github.com/en/actions/get-started/quickstart

https://github.com/actions/starter-workflows/blob/main/ci/django.yml

Continuous Deployment (Railway Deployment Documents & automated deployment)

https://docs.railway.com

https://docs.railway.com/deployments/github-autodeploys

Advanced Feature – Exporting Data as CSV

https://docs.djangoproject.com/en/5.0/howto/outputting-csv/

Arrow Icons

https://www.unicode.org/charts/PDF/U2190.pdf